LAB: Grayscale Image Segmentation - Gear

Date: 2025-MARCH-27

Author: HWANG MooYeol 22000814

Github: [-]

Demo Video: [-]

Introduction

1. Objective

- **Goal**: Defective Gear Inspection System

  Plastic gears are widely used in many applications, including toys, RC cars, and plastic-based hardware. Since they are made of plastic, it is fragile and can have broken gear teeth. You are asked to develop a machine vision system that can inspect defective plastic gears.

2. Preparation

Software Installation

- OpenCV 4.9.0, Visual Studio 2022

Algorithm

1. Overview

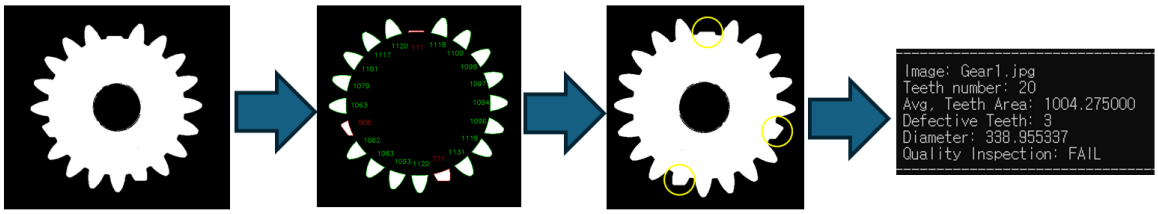

Figure 1. Overview

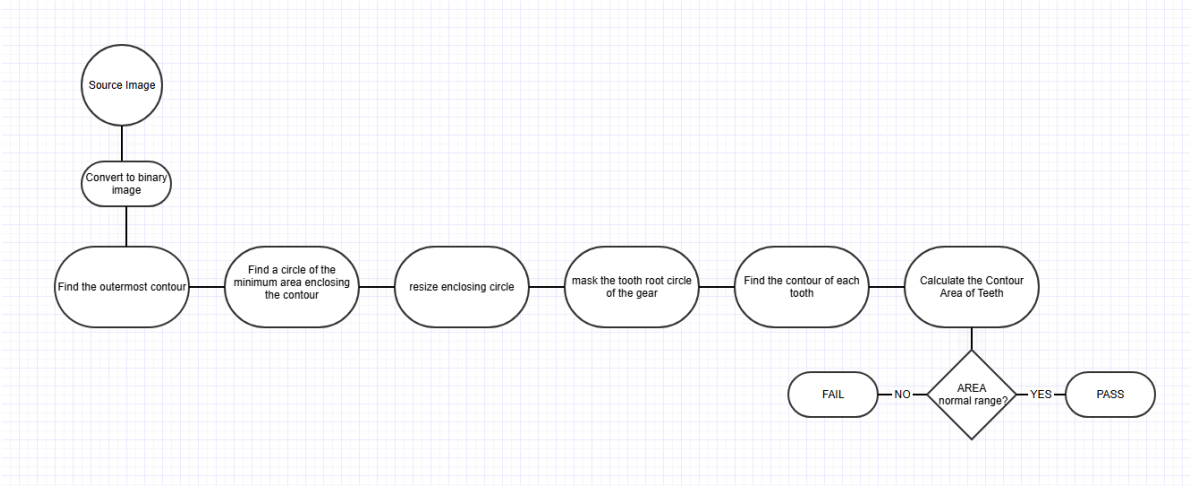

Figure 2. Flowchart

2. Procedure

- Design algorithms to detect defective gear for given images.
  - Include a flowchart to explain the algorithm flow.
- You should apply image processing algorithms, which you have learnt in class, as much as possible.
- If you want to use other algorithms that were not covered in class, then you should briefly explain how that algorithm works.
- For the output, you should calculate the following
  - Number of defective teeth
  - Diameter of the gear
  - Quality Inspection (Pass or Fail)
- You must explain each process with appropriate results
- You MUST include all the following in the report.
  - A simple flowchart to explain your algorithm
  - Apply appropriate filters to enhance image
  - Explain how the appropriate threshold value was chosen (if used)
  - Apply the appropriate morphology method to segment parts (if necessary)
    - Find the contour and draw the segmented objects. See Appendix
  - Explain how to determine the defective teeth

Thresholding and Morphology

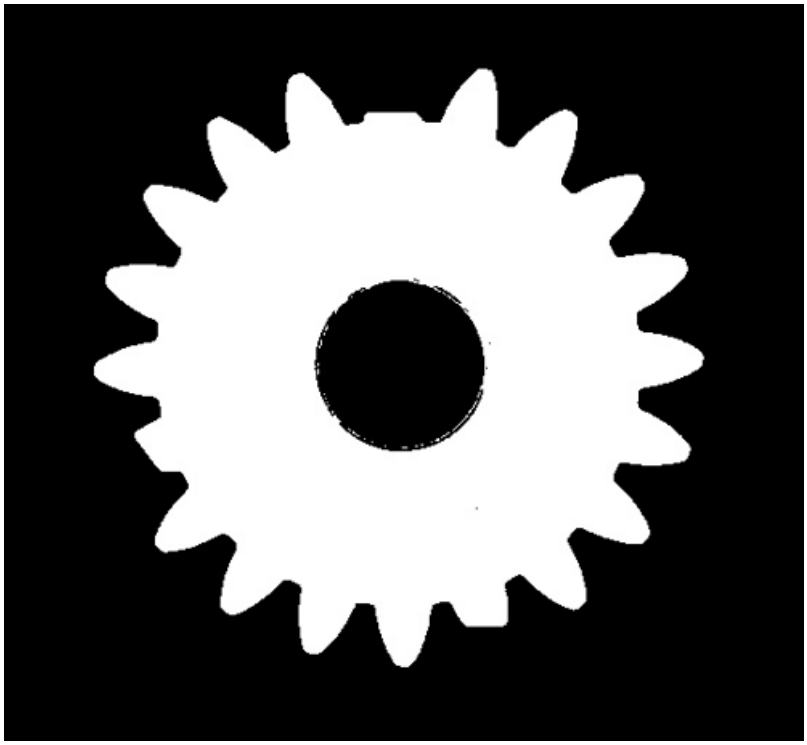

Figure 3. GIVEN image (Gear1)

Since the given image is black and white, the value of the Thresholding was set to 127, half of 255.

Used the `threshold` function to binarize source image.

```
cv::threshold(src, dst, thresh, maxval, type);

    src : Input image (must be a grayscale image)
    dst : Output image (thresholded binary image)
    thresh : Threshold value
    maxval : Maximum pixel value to assign if the condition is met
    type : Thresholding type
```

Image processing (Masking)

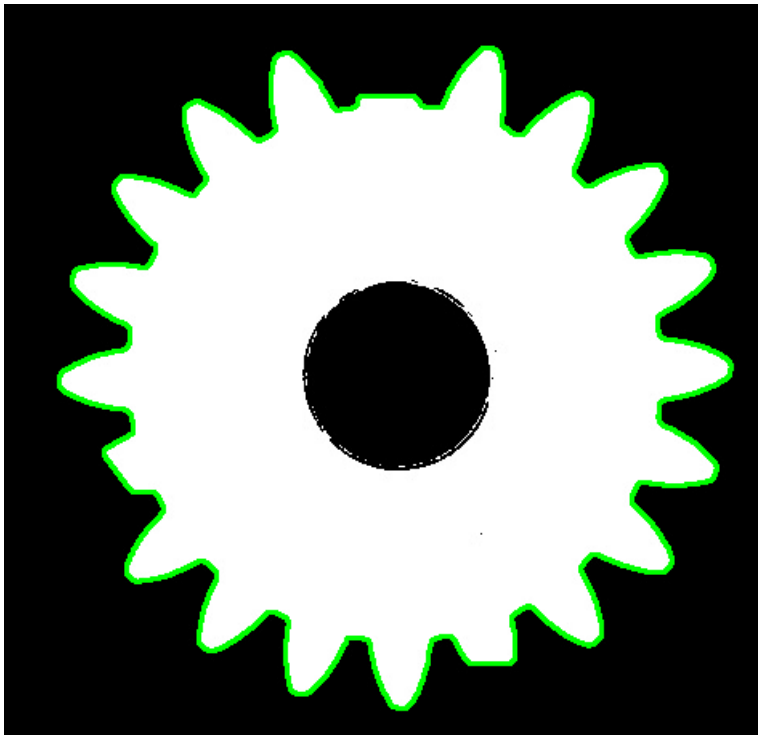

Figure 4. Contour for the gear (Gear1)

Found a contour for the whole outline to figure out the gears from the source image.

The `findContours` function was used to find contours.

```
cv::findContours(image, contours, hierarchy, mode, method);

    image : Input binary image (modifies this image, so use a copy if needed).
    contours : Vector of detected contours (each contour is a
std::vector<cv::Point>).
    hierarchy : Contour hierarchy (used when mode supports hierarchy).
    mode : Contour retrieval mode (how contours are retrieved).
    method : Contour approximation method (how contour points are stored).
```

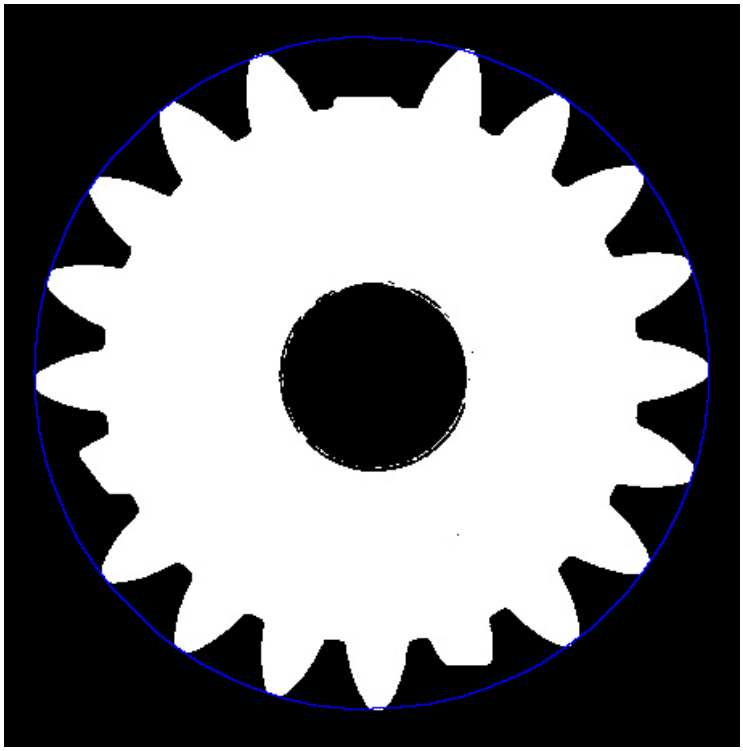

Figure 4. circumcircle for the gear (Gear1)

To find the center point and the gear root diameter, found a minimal circumcircle based on the entire contour.

`minEnclosingCircle` function was used to find the circumcircle.

```
cv::minEnclosingCircle(points, center, radius);

    points : A vector of cv::Point representing a contour.
    center : Output center point of the enclosing circle (cv::Point2f).
    radius : Output radius of the enclosing circle (float).
```

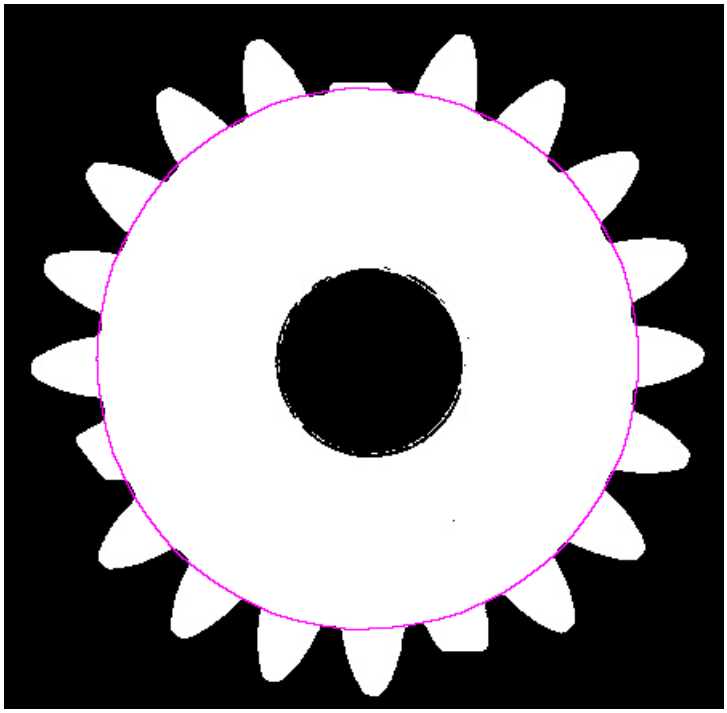

Figure 5. Root diameter for the gear (Gear1)

Adjusted the diameter of the gear circle to find the diameter of the gear root circle. We found the ratio of the gear tooth circle diameter as 0.805.

used the `circle` function

```
cv::circle(image, center, radius, color, thickness, lineType, shift);

    image : The image on which to draw the circle.
    center : The center point of the circle (cv::Point or cv::Point2f).
    radius : The radius of the circle (int or float).
    color : The color of the circle in BGR format (cv::Scalar).
    thickness : The thickness of the circle outline (if -1, the circle is
filled).
    lineType : The type of the line used to draw the circle.
    shift : Number of fractional bits in the center coordinates (used for sub-
pixel                    accuracy).
```

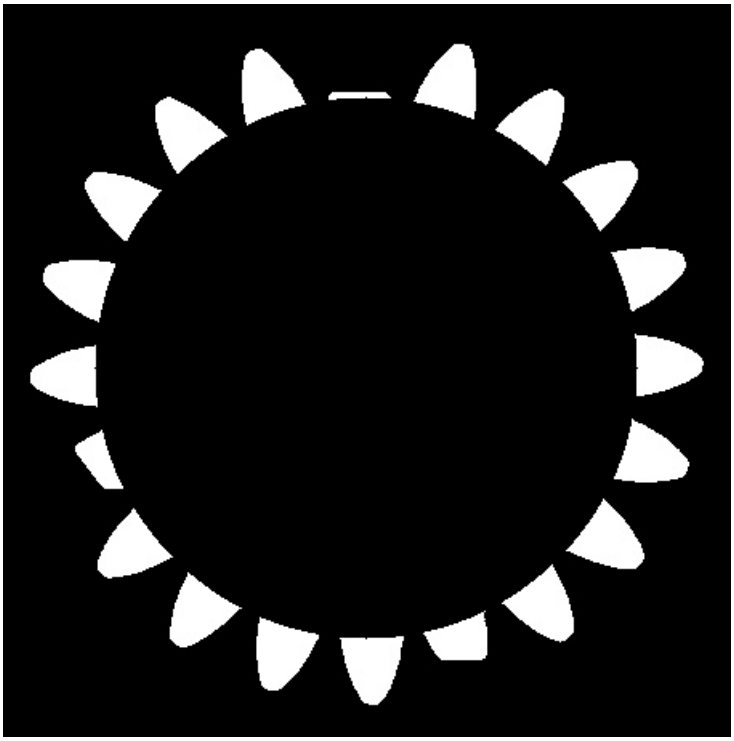

Figure 6. extracted gear teeth (Gear1)

Made masking with the diameter of the gear tooth root obtained. Through masking, gear teeth could be extracted from the original source image.

used the `bitwise_not` and `bitwise_and` function

```
cv::bitwise_not(input, output);

    input : The source image (typically a binary mask).
    output : The inverted image (bitwise NOT of the input).


cv::bitwise_and(image1, image2, output, mask);

    image1 : First input image (src).
    image2 : Second input image (src again in this case).
    output : The resulting image after AND operation.
    mask : The mask that defines which pixels to keep.
```

Determine the defective teeth

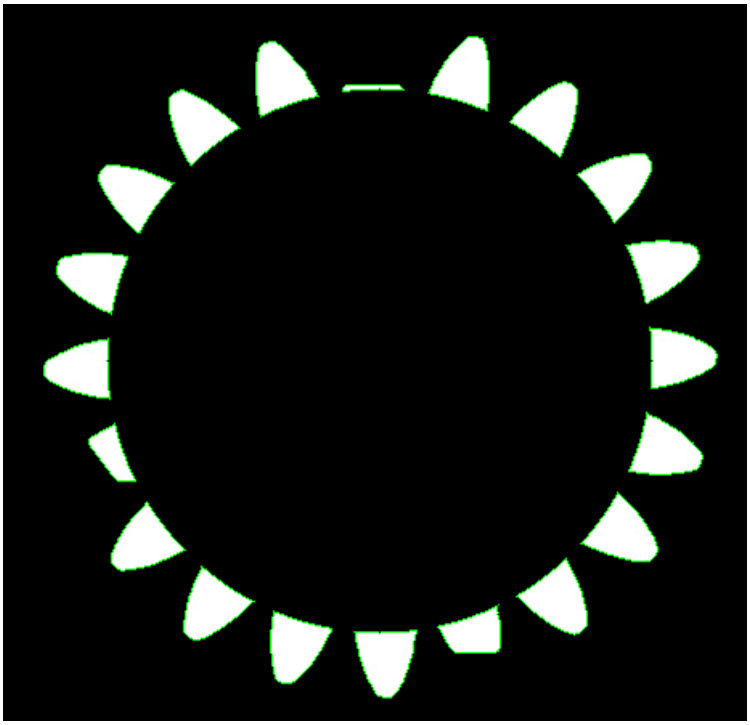

Figure 7. contour of teeths (Gear1)

Since the area of the teeth must be calculated to inspect, the contour for each tooth need to be found.

The `findContours` function was used to find contours.

```
findContours(imthres - mask, toothContours, hierarchy, RETR_EXTERNAL,
CHAIN_APPROX_SIMPLE);
```

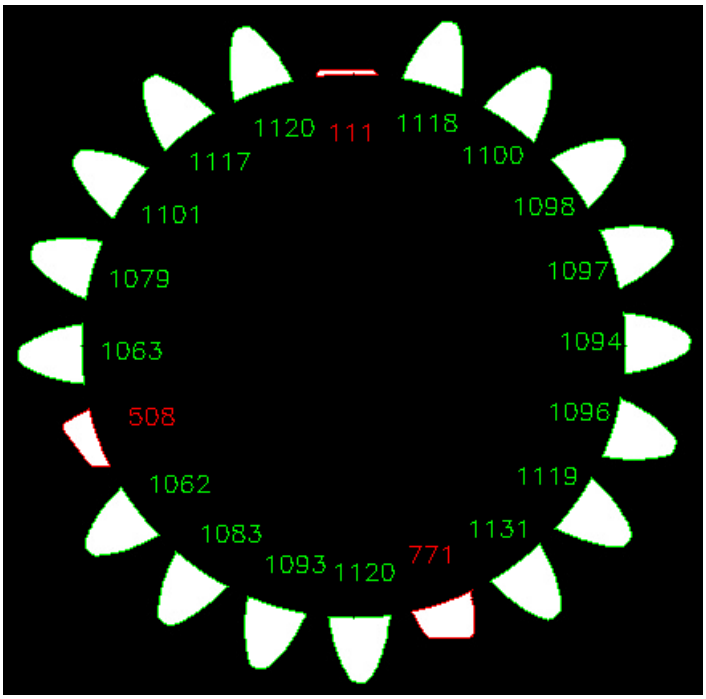

Figure 8. contour area of teeths (Gear1)

The area of the contour for each of the gear teeth was obtained, expressed in text, checked that the area was in the range of 1000 to 1500, and otherwise marked in red.

Function `contourArea` and `putText` was used.

```
double contourArea(InputArray contour, bool oriented = false);

    contour : A single contour, represented as a vector of cv::Point or
cv::Point2f.
    oriented (optional, default = false)
    If false : Returns the absolute area.
    If true : Returns a signed area, which helps determine the contour's
orientation                  (clockwise vs. counterclockwise).
Return Value
A double value representing the area of the contour.


void putText(Mat& img, const String& text, Point org, int fontFace,
             double fontScale, Scalar color, int thickness = 1,
             int lineType = LINE_8, bool bottomLeftOrigin = false);

    img : The image on which the text will be drawn.
    text : The text to be displayed.
    org : The bottom-left corner of the text in (x, y) coordinates.
    fontFace : The font style (e.g., FONT_HERSHEY_SIMPLEX).
    fontScale : The size of the text.
    color : The text color in Scalar(B, G, R).
    thickness : The thickness of the text stroke.
    lineType (optional, default = LINE_8) : Type of line (anti-aliased or
solid).
    bottomLeftOrigin (optional, default = false) : If true, the origin is the
bottom-left                                                    instead of the
top-left.
```

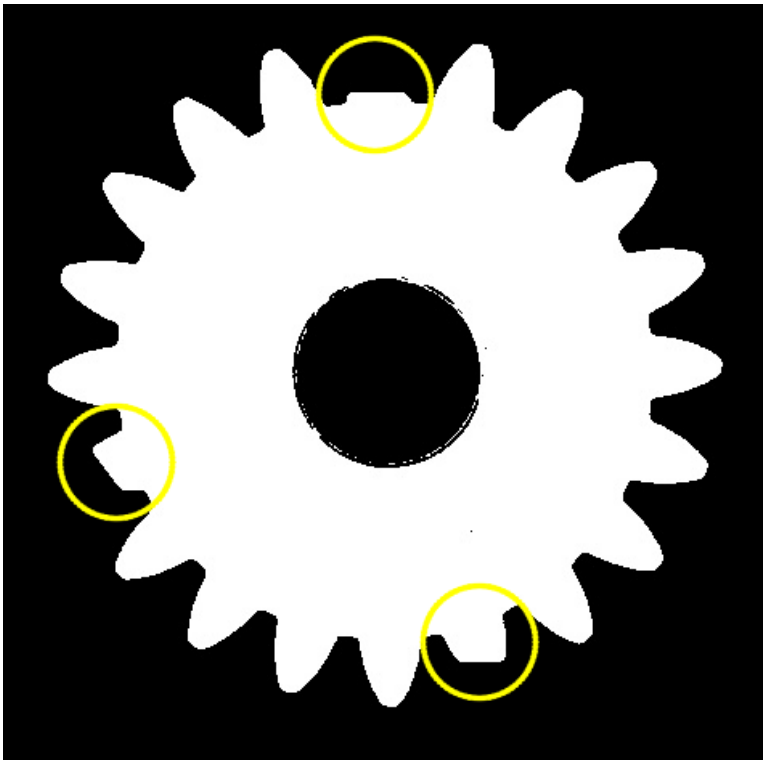

Figure 9. result of gear (Gear1)

To draw a circle on the teeth whose defects were identified, the center value for each tooth was found.

The `moments` function was used to find the center of the tooth contour

```
Moments moments(InputArray array, bool binaryImage = false);

    array : A contour or binary image (black & white) for which moments need to
be                  computed.
    binaryImage (optional, default = false)
        If true : Treats all nonzero pixels as 1 (useful for binary images).
        If false : Uses the actual pixel intensity values.

Return Value
Returns a cv::Moments object containing different moment values.
```

The `cv::Moments` structure contains **spatial** and **central moments**:

Spatial Moments

Name	Purpose
`m00`	**Area of the object**
`m10`	**Sum of x-coordinates (used for centroid)**
`m01`	**Sum of y-coordinates (used for centroid)**

Centroid Calculation

- The **centroid (center of mass)** of the contour is calculated as:

  Cx=m10/m00

  Cy=m01/m00

Central Moments (Shape Analysis)

- `mu20`, `mu02` → Measure **object elongation**.
- `mu11` → Measures **object orientation**.
- `nu20`, `nu02`, `nu11` → **Normalized central moments** (scale-invariant).

Result and Discussion

1. Final Result

In each pictures. The picture on the left is the result of the original picture showing defective teeth. The picture in the center is the result of showing the area values of the teeth. The picture on the right is the result of the analysis of the gear.

Gear 1

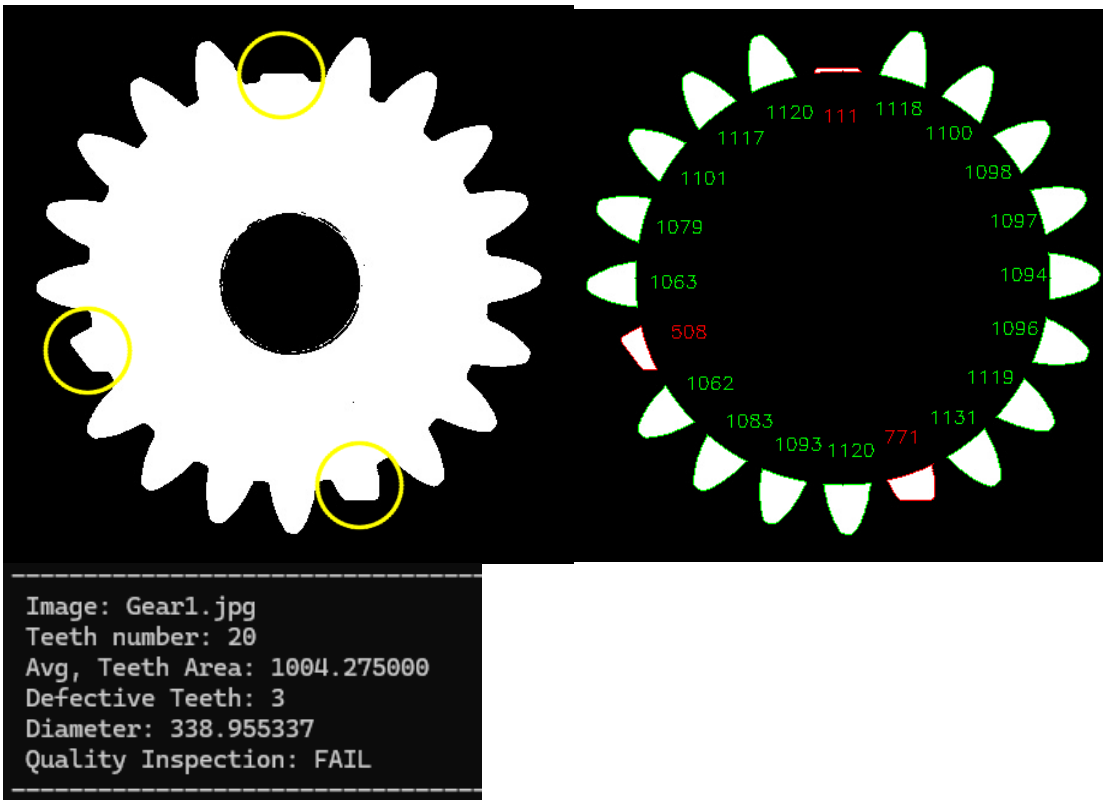

Figure 10. result for Gear1

Gear 2

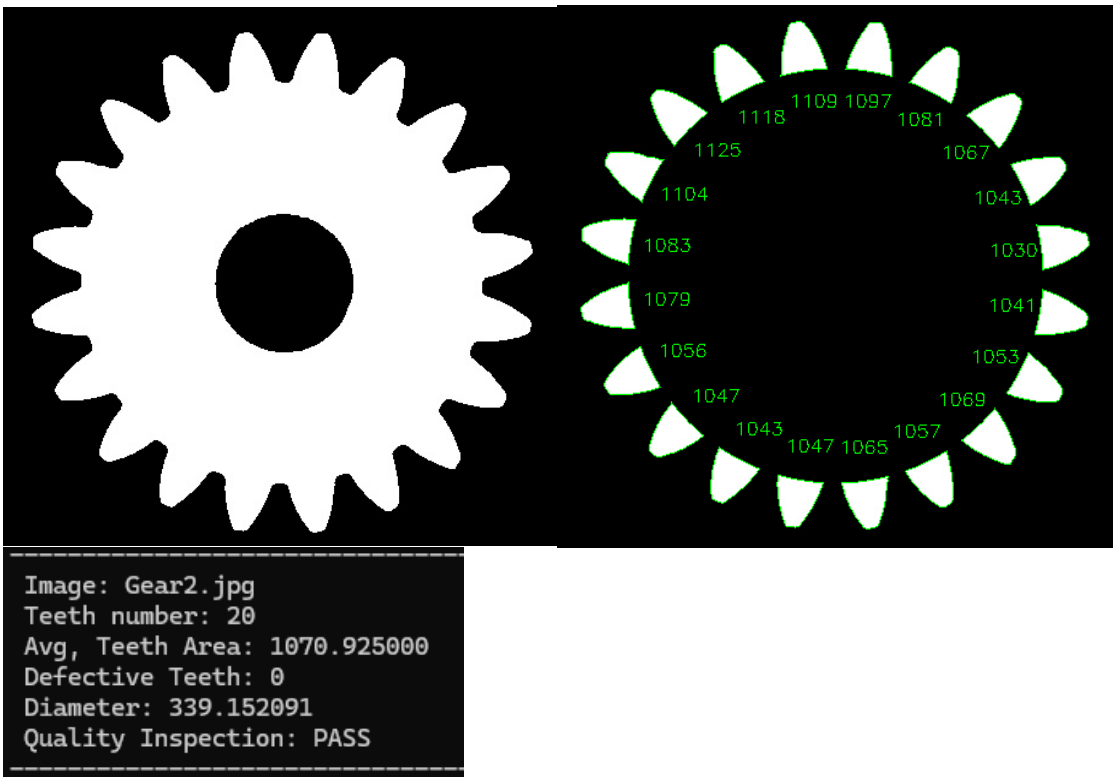

Figure 11. result for Gear2

Gear 3

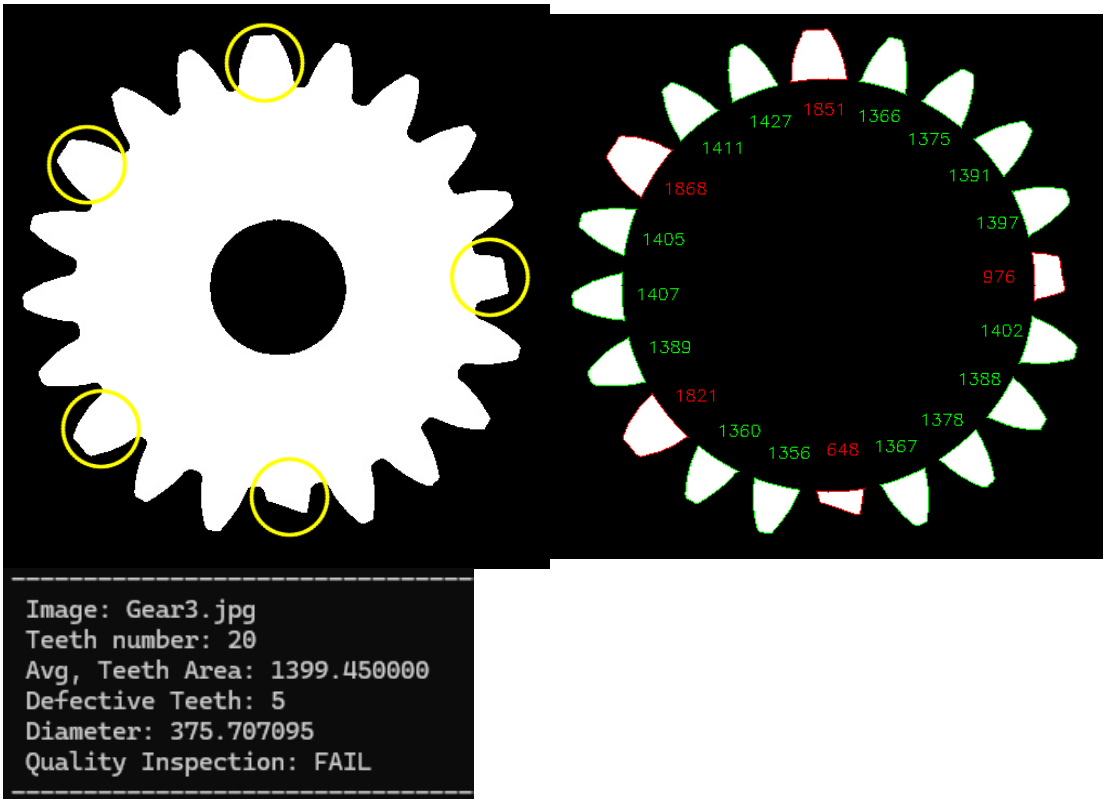

Figure 12. result for Gear3

Gear 4

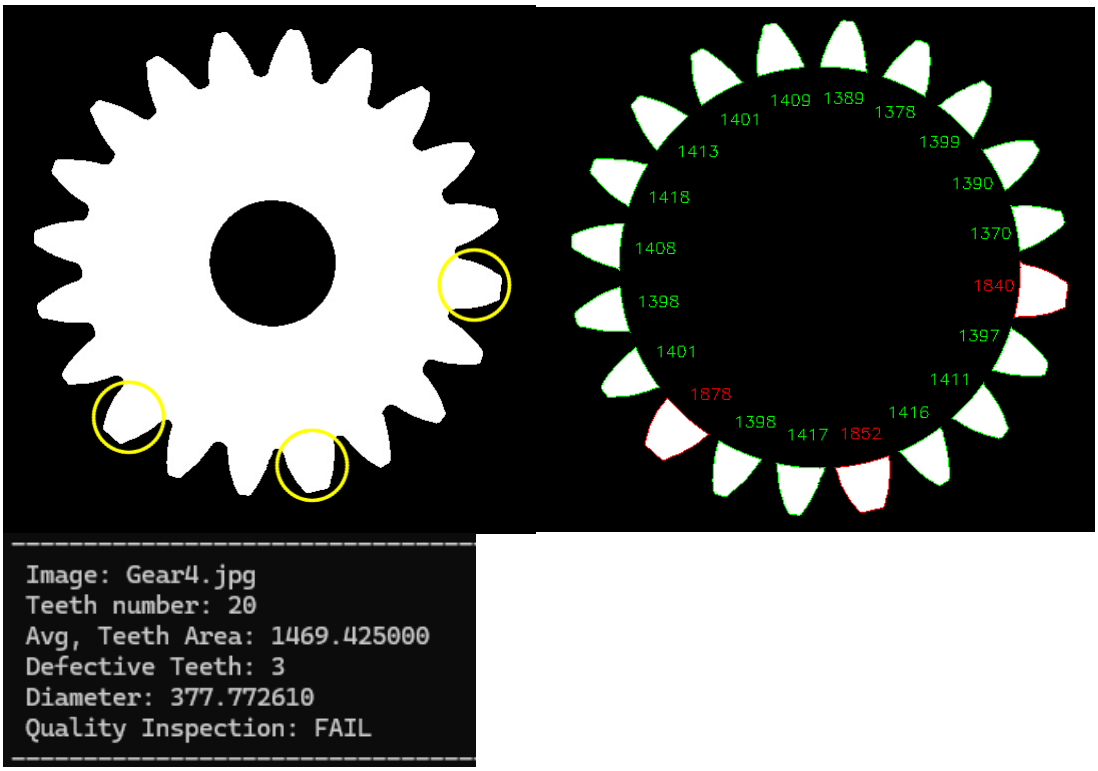

Figure 13. result for Gear4

2. Discussion

Gear #	Teeth nunbers	Defective Teeth	Quality
1	20	3	FAIL
2	20	0	PASS
3	20	5	FAIL
4	20	3	FAIL

It can be seen that the algorithm is well implemented so that all defects that have been visually identified can be found. The proposed algorithm has achieved the project goal successfully.

Conclusion

This project aims to develop a machine vision system for inspecting defective plastic gears, which are commonly used in toys, RC cars, and plastic-based hardware. Due to their fragile nature, plastic gears are prone to damage, such as broken or missing teeth. The system leverages image processing techniques to detect defects efficiently, ensuring quality control in manufacturing.

One of the main advantages of this approach is its adaptability. By adjusting parameters such as edge detection thresholds and contour analysis techniques, the system can be fine-tuned to suit different gear types and lighting conditions.

There are some limitations. The accuracy of the system depends on the image quality, and poor lighting conditions or occlusion can lead to false positives or speech. This is why the **preprocessing of images is important.**

Appendix

```
/* -------------------------------------------------------
 * Image Processing with Deep Learning
 * Created : 2025.03.21
 *------------------------------------------------------- */
#include <opencv2/opencv.hpp>
#include <iostream>

using namespace std;
using namespace cv;

int main()
{
    // List of gear images to be processed
    vector<String> imageFiles = { "Gear1.jpg", "Gear2.jpg", "Gear3.jpg",
"Gear4.jpg" };

    for (size_t idx = 0; idx < imageFiles.size(); idx++) {
        Mat src, src_gray, imthres, mask, result;
        src = imread(imageFiles[idx]); // Load the image

        cvtColor(src, src_gray, COLOR_BGR2GRAY); // Convert the image to
grayscale
        threshold(src_gray, imthres, 127, 255, THRESH_BINARY); // Apply binary
thresholding

        vector<vector<Point>> contours;
        vector<Vec4i> hierarchy;

        // Find outer contours of the gear
        findContours(imthres, contours, hierarchy, RETR_EXTERNAL,
CHAIN_APPROX_NONE);

        Point2f center;
        float radius;
```

```
        minEnclosingCircle(contours[0], center, radius); // Find the minimum
enclosing circle of the gear

        // Create a mask to isolate the gear's teeth
        mask = Mat::zeros(src.size(), CV_8UC1);
        circle(mask, center, radius * 0.805, Scalar(255), -1); // Draw a circle
inside the gear to mask the center

        Mat mask_inv;
        bitwise_not(mask, mask_inv); // Invert the mask
        bitwise_and(src, src, result, mask_inv); // Apply the inverted mask to
the source image

        vector<vector<Point>> toothContours;
        // Find contours of the teeth by subtracting the mask from the binary
image
        findContours(imthres - mask, toothContours, hierarchy, RETR_EXTERNAL,
CHAIN_APPROX_SIMPLE);

        // Variables for quality inspection
        string Quality_Inspection = "PASS";
        int Defective = 0;
        double avg_area = 0;

        for (size_t i = 0; i < toothContours.size(); i++) {
            double tooth_area = contourArea(toothContours[i]); // Compute the
area of each tooth
            avg_area += tooth_area;
            Moments m = moments(toothContours[i]); // Compute moments for each
tooth

            // Set color based on whether the tooth's area is within the
acceptable range
            Scalar color = (tooth_area <= 1000 || tooth_area >= 1500) ?
Scalar(0, 0, 255) : Scalar(0, 255, 0);

            drawContours(result, toothContours, (int)i, color, 1); // Draw the
detected tooth contours
            double ratio = 0.23; // Scaling factor for text placement

            if (m.m00 != 0) {
                int cx = int(m.m10 / m.m00); // Compute the x-coordinate of the
centroid
                int cy = int(m.m01 / m.m00); // Compute the y-coordinate of the
centroid

                string area_text = to_string(int(tooth_area)); // Convert area
to string
                int baseline = 0;
                Size text_size = getTextSize(area_text, FONT_HERSHEY_SIMPLEX,
0.5, 1.5, &baseline);

                // Display the tooth area near the centroid
                putText(result, area_text, Point((center.x) * ratio + (cx -
text_size.width / 2) * (1 - ratio),
                    (center.y) * ratio + (cy + text_size.height / 2) * (1 -
ratio)),
                    FONT_HERSHEY_SIMPLEX, 0.5, color, 1.5);
```

```
                // Mark defective teeth if the area is out of the acceptable
range
                if (tooth_area <= 1000 || tooth_area >= 1500) {
                    circle(src, Point(cx, cy), 35, Scalar(0, 255, 255), 2,
LINE_AA);
                    Quality_Inspection = "FAIL";
                    Defective += 1;
                }
            }
        }

        avg_area = avg_area / toothContours.size(); // Compute the average tooth
area

        // Print inspection results
        printf("-------------------------------------------------------------------
------------------------------\n");
        printf(" Image: %s\n Teeth number: %zu\n Avg, Teeth Area: %f\n Defective
Teeth: %d\n Diameter: %f\n Quality Inspection: %s\n",
            imageFiles[idx].c_str(), toothContours.size(), avg_area, Defective,
radius * 0.805 * 2, Quality_Inspection.c_str());
        printf("-------------------------------------------------------------------
------------------------------\n");

        // Display the original and processed images
        imshow("Original " + to_string(idx + 1), src);
        imshow("Result " + to_string(idx + 1), result);
    }

    waitKey(0); // Wait for a key press before closing windows
    return 0;
}
```